# 11. State in React

## ▼ What is the State?

- State is a plain JavaScript object used by React to represent information about the component's current situation.
- OR
- A state is an object that holds data and information related to React component.
- It can be used to store, manage, and update data within the application, which in turn allows for dynamic changes to user interfaces.
- It is mutable (can change over time)
- When the state changes, `React re-renders` the component to reflect the updated data in the UI.
- Think of the state as a **reactive memory** (**brain**🧠) for your component.

---

## ▼ Why Do We Need State?

- We need state in React because:

| Reason | Description |
|---|---|
| ✅ Dynamic UI | State lets us build **interactive, real-time updating components**. |
| ✅ Renders Based on Data | UI changes automatically when the state changes. |
| ✅ Stores Local Data | It helps store data that is **local to a component** (like input field value, counter, etc.). |
| ✅ Essential for Interactivity | Without state, components would be static. |

---

## ▼ How to Initialize and Use State

### ▼ 1. Class-Based Component with `this.state`

- In a **class-based component**, the state is an object that holds **data or information** about that component.

- It is local to the component.
- When the state changes using the `setState()` methods, the **component re-renders automatically**, and the UI updates to reflect the new state.
- State makes your class component **interactive and dynamic**.

## ▼ Syntax: How to Define State in Class Components

- **Syntax:**

```
class ComponentName extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      // key-value pairs of data
      key: initialValue,
    };
  }
}
```

- The `constructor()` is used to initialize the state.
- The `super(props)` must be called before accessing `this`.

## ▼ What is `setState()`?

- `setState()` is a **built-in method** in React used to **update** the component's state.
- You **should never modify the state directly** using `this.state.key = value`.
- **Wrong:**

```
this.state.count = 10; // ❌ Never directly modify state
```

- **Correct:**

```
this.setState({ count: 10 }); // ✅ Use setState
```

- When you use `setState()`, React **merges the new value with the old state and then triggers a re-render**.

## ▼ Example of State in a Class Component

- **Code:**

```
import React, { Component } from "react";

export default class Counter extends Component {
  constructor() {
    super();
    this.state = {
      count: 0, // intial state
    };
  }

  //increment function which will update the count
  increment = () => {
    this.setState({ count: this.state.count + 1 }); // updating state
  };

  render() {
    return (
      <>
        <h1>Count: {this.state.count}</h1>
        <button onClick={this.increment}>Increase</button>
      </>
    );
  }
}
```

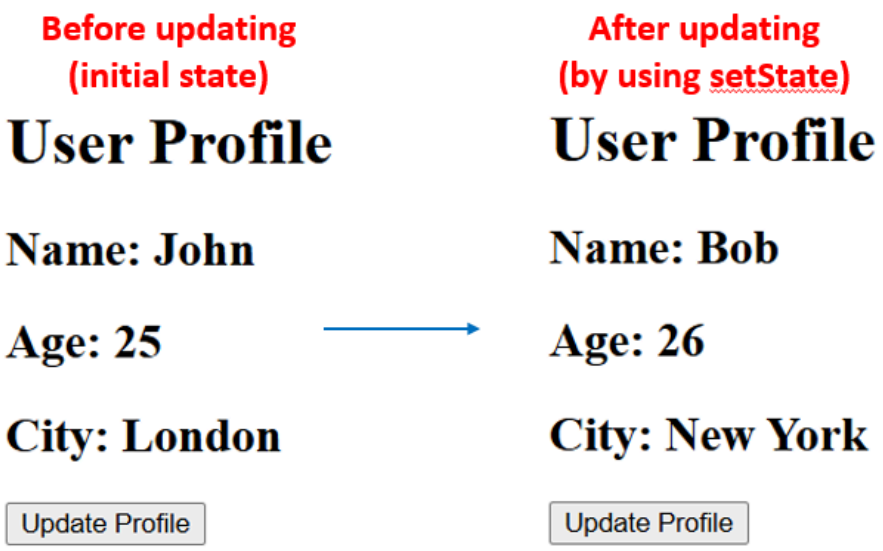
## ▼ How to Update Multiple States in Class-Based Components

- In a class-based component, `this.state` is an **object**, so it can contain multiple key-value pairs.
- To **update multiple values** in the state at the same time, you use a **single call to** `this.setState()`, passing an object with all the keys you want to update.
- **For Example →**

```
import React, { Component } from "react";

class UserProfile extends Component {
  constructor() {
    super();
    this.state = {
      name: "John",
      age: 25,
      city: "London",
    };
  }

  updateProfile = () => {
    this.setState({
      name: "Bob",
      age: 26,
      city: "New York",
    });
  };

  render() {
    return (
      <>
        <h1>User Profile</h1>
        <h2>Name: {this.state.name}</h2>
        <h2>Age: {this.state.age}</h2>
        <h2>City: {this.state.city}</h2>
        <button onClick={this.updateProfile}>Update Profile</button
>
      </>
```

```
    );
  }
}

export default UserProfile;
```

- **Output: After clicking on the button**




---

## ▼ 2. Functional Component with `useState` Hook

- In functional components, state is managed using the **hooks** (`useState()`) in React, which was introduced in **React v16.8**
- `useState` lets you create and update state inside a function, making functional components capable of holding dynamic data and re-rendering the UI on changes just like class components.

### ▼ Syntax of `useState()`

```
const [state, setState] = useState(initialValue);
```

| Part | Description |
|---|---|
| `state` | Current state value |
| `setState` | Function to update the state |

| Part | Description |
| --- | --- |
| `useState()` | Hook that returns a pair: current value and setter function |
| `initialValue` | The initial value of the state (can be any data type) |

- When you console the `useState` hook, you will get an array with two values.

```
console.log(useState()); //array
```

- **Output:**

```
▼ (2) [undefined, ƒ] ℹ
    0: undefined
  ▶ 1: ƒ ()
    length: 2
  ▶ [[Prototype]]: Array(0)
```

- The first one is the state, and the second one is the function that is responsible for updating the state.

---

# ▼ What are React Hooks?

- React **Hooks** let you **use state and other React features** in **functional components**, without writing class components.
- But they must follow **specific rules** to work correctly. These are called the **Rules of Hooks**.

## ▼ Rules of Hooks (Core Rules)

1. **Rule-1: Only Call Hooks at the Top Level**
   - Hooks **must be called at the top level** of your React function - **not inside loops, conditions, or nested functions**.
   - **Incorrect:**

```
if (condition) {
  const [count, setCount] = useState(0); // ❌ Don't do this!
```

```
}
```

- **Correct:**

```
const [count, setCount] = useState(0); // ✅ Always call hooks directly
```

2. **Rule-2: Only Call Hooks from React Functions**
   - Hooks can only be used **inside functional components or custom hooks** - **not in regular JavaScript functions** or class components.
   - **Invalid:**

```
function helperFunction() {
  const [count, setCount] = useState(0); // ❌ This will throw an error
}
```

   - **Correct:**

```
function MyComponent() {
  const [count, setCount] = useState(0); // ✅ Only inside React components
}
```

3. **Rule-3: Always use `use` prefix when naming custom hooks** (Not Official rule but highly recommended)
   - Example: `useFetch`, `useAuth`, `useTheme`
   - Use hooks first, then apply logic based on the result.

---

## ▼ Example

- **Code:**

```
import React, { useState } from "react";

const Counter = () => {
```

```
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1); // updates state
  };

  return (
    <>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increase</button>
    </>
  );
};

export default Counter;
```

---

## ▼ Updating Multiple States in Functional Components

- **Code:**

```
import React, { useState } from "react";

const UserProfile = () => {
  const [name, setName] = useState("John");
  const [age, setAge] = useState(25);
  const [city, setCity] = useState("London");

  const updateProfile = () => {
    setName("Bob");
    setAge(26);
    setCity("New York");
  };

  return (
    <>
      <h1>User Profile</h1>
      <h2>Name: {name}</h2>
```

```
      <h2>Age: {age}</h2>
      <h2>City: {city}</h2>
      <button onClick={updateProfile}>Update Profile</button>
    </>
  );
};

export default UserProfile;
```